B.Bhanu Prasad

Batch no:38

2303a52021

Task Description -1 – (Stack Using AI Guidance)

• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

• A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

```python
# generate a code which perform basic stack operations with proper documentation
class Stack:
    """
    A class to represent a stack data structure.

    Attributes
    ----------
    items : list
        A list to store the elements of the stack.

    Methods
    -------
    push(item):
        Adds an item to the top of the stack.
    pop():
        Removes and returns the item at the top of the stack.
    peek():
        Returns the item at the top of the stack without removing it.
    is_empty():
        Returns True if the stack is empty, False otherwise.
    size():
        Returns the number of items in the stack.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Parameters
        ----------
        item : any
            The item to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """
        Remove and return the item at the top of the stack.

        Returns
        -------
        any
            The item at the top of the stack.

        Raises
        ------
        IndexError
            If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.items.pop()

    def peek(self):
        """
        Return the item at the top of the stack without removing it.

        Returns
        -------
        any
            The item at the top of the stack.
```

```python
        Raises
        ------
        IndexError
            If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.

        Returns
        -------
        bool
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0

    def size(self):
        """
        Return the number of items in the stack.

        Returns
        -------
        int
            The number of items in the stack.
        """
        return len(self.items)
```

```python
        return len(self.items)
# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Top item:", stack.peek())  # Output: Top item: 3
    print("Stack size:", stack.size())  # Output: Stack size: 3
    print("Popped item:", stack.pop())  # Output: Popped item: 3
    print("Stack is empty:", stack.is_empty())  # Output: Stack is empty: False
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 11.2.py"
Top item: 3
Stack size: 3
Popped item: 3
Stack is empty: False
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Task Description -2 – (Queue Design)

• Task:Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

• A complete Queue implementation including enqueue, dequeue,

front element access, and size calculation.

```python

# generate queue data structure with basic operations and proper documentation
class Queue:
    """
    A class to represent a queue data structure.

    Attributes
    ----------
    items : list
        A list to store the elements of the queue.

    Methods
    -------
    enqueue(item):
        Adds an item to the end of the queue.
    dequeue():
        Removes and returns the item at the front of the queue.
    peek():
        Returns the item at the front of the queue without removing it.
    is_empty():
        Returns True if the queue is empty, False otherwise.
    size():
        Returns the number of items in the queue.
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """
        Add an item to the end of the queue.

        Parameters
        ----------
        item : any
            The item to be added to the queue.
        """
        self.items.append(item)
```

```python

    def dequeue(self):
        """
        Remove and return the item at the front of the queue.

        Returns
        -------
        any
            The item at the front of the queue.

        Raises
        ------
        IndexError
            If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.items.pop(0)

    def peek(self):
        """
        Return the item at the front of the queue without removing it.

        Returns
        -------
        any
            The item at the front of the queue.

        Raises
        ------
        IndexError
            If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.items[0]

```

```python

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns
        -------
        bool
            True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0

    def size(self):
        """
        Return the number of items in the queue.

        Returns
        -------
        int
            The number of items in the queue.
        """
        return len(self.items)
# Example usage
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Front item:", queue.peek())  # Output: Front item: 1
    print("Queue size:", queue.size())  # Output: Queue size: 3
    print("Dequeued item:", queue.dequeue())  # Output: Dequeued item: 1
    print("Queue is empty:", queue.is_empty())  # Output: Queue is empty: False
```

Output:

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 11.2.py"
Front item: 1
Queue size: 3
Dequeued item: 1
Queue is empty: False
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Ln 189, Col 12 Spaces: 4 UTF-8 CRLF { } Python 3.14.2

Utilize AI to build a singly linked list supporting insertion

and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion logic, and display functionality.

```python
# generate a code for singly linked list for insertion and traversal with proper documentation
class Node:
    """
    A class to represent a node in a singly linked list.

    Attributes
    ----------
    data : any
        The data stored in the node.
    next : Node
        A reference to the next node in the list.

    Methods
    -------
    __init__(data):
        Initializes a new node with the given data and sets next to None.
    """
    def __init__(self, data):
        """
        Initialize a new node with the given data and sets next to None.

        Parameters
        ----------
        data : any
            The data to be stored in the node.
        """
        self.data = data
        self.next = None
```

```python
        self.next = None
class SinglyLinkedList:
    """
    A class to represent a singly linked list.

    Attributes
    ----------
    head : Node
        A reference to the first node in the list.

    Methods
    -------
    insert(data):
        Inserts a new node with the given data at the end of the list.
    traverse():
        Traverses the list and returns a list of all node data.
    """
    def __init__(self):
        """Initialize an empty singly linked list."""
        self.head = None

    def insert(self, data):
        """
        Insert a new node with the given data at the end of the list.

        Parameters
        ----------
        data : any
            The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
```

```python

    def traverse(self):
        """
        Traverse the list and return a list of all node data.

        Returns
        -------
        list
            A list containing the data of all nodes in the linked list.
        """
        elements = []
        current_node = self.head
        while current_node:
            elements.append(current_node.data)
            current_node = current_node.next
        return elements
# Example usage
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List Elements:", linked_list.traverse())  # Output: Linked List Elements: [10, 20, 30]
```

OUTPUT:

```
PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 11.2.py"
Linked List Elements: [10, 20, 30]
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Task: Implement a Binary Search Tree with AI support focusing

on insertion and traversal.

Expected Output:

• BST program with correct node insertion and in-order traversal

output.

```python
# genearte a code for Binary Search Tree with insertion and inorder traversal with proper documentation
class TreeNode:
    """
    A class to represent a node in a binary search tree.

    Attributes
    ----------
    data : any
        The data stored in the node.
    left : TreeNode
        A reference to the left child node.
    right : TreeNode
        A reference to the right child node.

    Methods
    -------
    __init__(data):
        Initializes a new tree node with the given data and sets left and right to None.
    """
    def __init__(self, data):
        """
        Initialize a new tree node with the given data and sets left and right to None.

        Parameters
        ----------
        data : any
            The data to be stored in the node.
        """
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    """
    A class to represent a binary search tree.

    Attributes
    ----------
    root : TreeNode
        A reference to the root node of the tree.

    Methods
    -------
    insert(data):
        Inserts a new node with the given data into the tree while maintaining the BST property.
    inorder_traversal():
        Performs an inorder traversal of the tree and returns a list of node data in sorted order.
    """
```

```python
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, data):
        """
        Insert a new node with the given data into the tree while maintaining the BST property.

        Parameters
        ----------
        data : any
            The data to be stored in the new node.
        """
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def inorder_traversal(self):
        """
        Perform an inorder traversal of the tree and return a list of node data in sorted order.

        Returns
        -------
        list
            A list containing the data of all nodes in the binary search tree in sorted order.
        """
        elements = []
        self._inorder_recursive(self.root, elements)
        return elements

    def _inorder_recursive(self, node, elements):
        if node:
            self._inorder_recursive(node.left, elements)
            elements.append(node.data)
```

```python

    def _inorder_recursive(self, node, elements):
        if node:
            self._inorder_recursive(node.left, elements)
            elements.append(node.data)
            self._inorder_recursive(node.right, elements)
# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("Inorder Traversal:", bst.inorder_traversal())  # Output: Inorder Traversal: [2, 3, 4, 5, 6, 7, 8]
```

OUTPUT:

```
> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 11.2.py"
Inorder Traversal: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Task Description -5 – (Hash Table Implementation)

• Task: Create a hash table using AI with collision handling

Expected Output:

• Hash table supporting insert, search, and delete using chaining

```python
# create a hash table which handle collisions using chaining with proper documentation
class HashTable:
    """
    A class to represent a hash table data structure that handles collisions using chaining.

    Attributes
    ----------
    size : int
        The size of the hash table.
    table : list
        A list of lists to store the key-value pairs in case of collisions.

    Methods
    -------
    __init__(size):
        Initializes the hash table with the given size and creates an empty table.
    _hash(key):
        Computes the hash value for a given key.
    insert(key, value):
        Inserts a key-value pair into the hash table, handling collisions using chaining
    get(key):
        Retrieves the value associated with a given key from the hash table.
    remove(key):
        Removes the key-value pair associated with a given key from the hash table.
        """
    def __init__(self, size=10):
        """
        Initialize the hash table with the given size and creates an empty table.

        Parameters
        ----------
        size : int, optional
            The size of the hash table (default is 10).
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """
        Compute the hash value for a given key.

        Parameters
        ----------
        key : any
            The key to be hashed.

```

```python
        Returns
        -------
        int
            The hash value of the key.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table, handling collisions using chaining.

        Parameters
        ----------
        key : any
            The key to be inserted.
        value : any
            The value associated with the key.
        """
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))

    def get(self, key):
        """
        Retrieve the value associated with a given key from the hash table.

        Parameters
        ----------
        key : any
            The key whose value is to be retrieved.

        Returns
        -------
        any
            The value associated with the given key.

        Raises
        ------
        KeyError
            If the key is not found in the hash table.
        """
```

Ln 519, Col 55 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Go Live

```python
        Remove a key-value pair from the hash table.

        Parameters
        ----------
        key : any
            The key of the pair to be removed.

        Raises
        ------
        KeyError
            If the key is not found in the hash table.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index].pop(i)
                return
        raise KeyError(f"Key '{key}' not found in hash table")
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print("Name:", hash_table.get("name"))  # Output: Name: Alice
    print("Age:", hash_table.get("age"))    # Output: Age: 30
    print("City:", hash_table.get("city"))  # Output: City: New York

    hash_table.remove("age")

    try:
        print(hash_table.get("age"))  # This will raise a KeyError
    except KeyError as e:
        print(e)  # Output: Key 'age' not found in hash table
```

OUTPUT:

```
> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 11.2.py"
Name: Alice
Age: 30
City: New York
"Key 'age' not found in hash table"
PS C:\Users\Bhanu Prasad\AIAC LAB>
```